

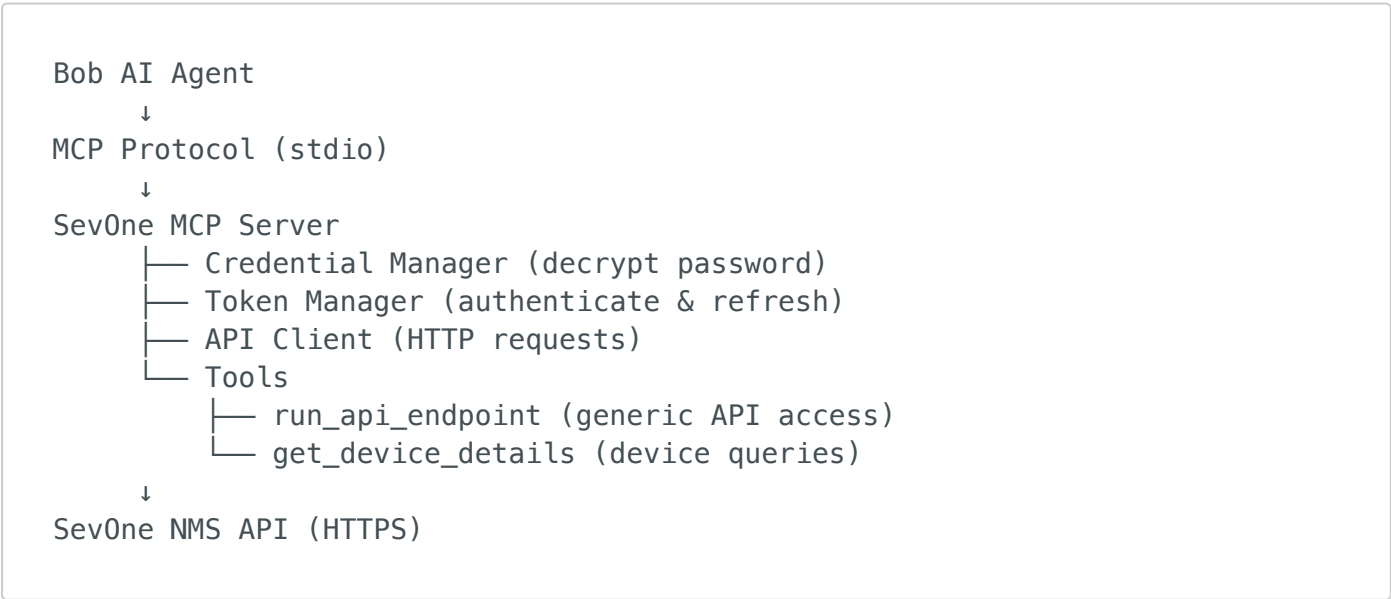
SevOne MCP Server

A Model Context Protocol (MCP) server that provides secure, authenticated access to the SevOne REST API. This server features encrypted credential storage, automatic token management, and a generic API endpoint execution tool.

Features

- 🔑 **Secure Credential Management:** Passwords encrypted at rest using Fernet symmetric encryption
- 🔄 **Automatic Token Management:** Handles authentication and token refresh transparently
- 🔧 **Multiple Tools:** Generic API tool plus specialized tools for common operations
- 🤖 **Custom Bob Mode:** Natural language interface to SevOne API (SevOne API Expert mode)
- 📝 **Comprehensive Logging:** Detailed logging for debugging and audit trails
- ⚡ **Async/Await:** Built with modern async Python for optimal performance

Architecture



Installation

Prerequisites

- Python 3.10 or higher
- Access to a SevOne NMS instance
- SevOne API credentials (username and password)

Setup

1. Clone the repository

```
git clone <repository-url>  
cd sevone-mcp-server
```

2. Create virtual environment

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Generate encryption key and encrypt password

```
python scripts/encrypt_password.py
```

This script will:

- Generate a new encryption key (or use an existing one)
- Encrypt your SevOne password
- Display the values to add to your **.env** file

5. Create **.env** file

```
cp .env.example .env
```

Edit `.env` with your values:

```
SEVONE_HOSTNAME=your-sevone-hostname.com
SEVONE_USERNAME=your-username
SEVONE_PASSWORD_ENCRYPTED=<encrypted-password-from-script>
SEVONE_ENCRYPTION_KEY=<encryption-key-from-script>
SEVONE_VERIFY_SSL=false
LOG_LEVEL=INFO
```

Usage

Running the Server

```
python src/server.py
```

The server will:

1. Load and decrypt credentials from `.env`
2. Authenticate with SevOne using `/api/v3/users/signin`
3. Start the MCP server on stdio
4. Wait for tool invocations from Bob AI

Configuring Bob AI

Add to your Bob MCP configuration file:

```
{
  "mcpServers": {
    "sevone-api": {
      "command": "python",
      "args": ["/path/to/sevone-mcp-server/src/server.py"],
      "env": {
        "PYTHONPATH": "/path/to/sevone-mcp-server"
      }
    }
  }
}
```

Available Tools

The server provides multiple tools for interacting with SevOne:

1. `run_api_endpoint` (Generic API Access)

Execute any SevOne API endpoint directly.

2. `get_device_details` (Device Queries)

Convenient tool for querying device information. See [GET_DEVICE_DETAILS_TOOL.md](#) for detailed documentation.

3. `get_device_group_details` (Device Group Queries)

Query device group information by IDs or hierarchical paths. See [GET_DEVICE_GROUP_DETAILS_TOOL.md](#) for detailed documentation.

4. `get_device_metadata_ids` (Metadata ID Lookup)

Get metadata namespace and attribute IDs for devices. See [GET_DEVICE_METADATA_IDS_TOOL.md](#) for detailed documentation.

5. `add_devicegroup_rules` (Device Group Rule Creation)

Create device group rules that automatically assign devices to groups based on various criteria. See [ADD_DEVICEGROUP_RULES_TOOL.md](#) for detailed documentation.

Quick Example:

```
{
  "tool": "add_devicegroup_rules",
  "arguments": {
    "device_group_id": 224,
    "namespace_id": 14,
    "attribute_id": 185,
    "metadata_value_expression": "WiFi Access Point"
  }
}
```

Quick Example:

```
{
  "tool": "get_device_details",
  "arguments": {
    "device_names": ["router1", "switch2"],
    "include_metadata": true
  }
}
```

Tool Details

run_api_endpoint Tool

Tool Parameters

- **endpoint** (required): API endpoint path (e.g., `/api/v3/devices`)
- **method** (required): HTTP method (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`)
- **data** (optional): Request body for POST/PUT/PATCH requests
- **params** (optional): Query parameters for GET requests

Examples

List Devices

```
{
  "tool": "run_api_endpoint",
  "arguments": {
    "endpoint": "/api/v3/devices",
    "method": "GET",
    "params": {
      "limit": 10
    }
  }
}
```

Create Device

```
{
  "tool": "run_api_endpoint",
  "arguments": {
    "endpoint": "/api/v3/devices",
    "method": "POST",
    "data": {
      "name": "Router-01",
      "ip": "192.168.1.1",

```

```
        "deviceTypeId": 1,
        "peerId": 1
    }
}
```

Update Device Metadata

```
{
  "tool": "run_api_endpoint",
  "arguments": {
    "endpoint": "/api/v3/entity/DEVICE/id/123/metadata",
    "method": "PUT",
    "data": {
      "metadata": [
        {
          "namespaceId": 1,
          "attributeId": 5,
          "value": "Production"
        }
      ]
    }
  }
}
```

Get Device Details

```
{
  "tool": "run_api_endpoint",
  "arguments": {
    "endpoint": "/api/v3/devices/123",
    "method": "GET"
  }
}
```

Project Structure

```
sevone-mcp-server/
├── src/
│   ├── __init__.py
│   ├── server.py                # Main MCP server
│   ├── auth/
│   │   ├── __init__.py
│   │   ├── credentials.py      # Credential encryption/decryption
│   │   └── token_manager.py    # Token lifecycle management
│   └── api/
```

```

├── __init__.py
├── client.py                # SevOne API client
├── tools/
│   ├── __init__.py
│   ├── run_api_endpoint.py    # Generic API tool
│   ├── get_device_details.py  # Device query tool
│   ├── get_device_group_details.py # Device group query tool
│   ├── get_device_metadata_ids.py # Metadata ID lookup tool
│   └── add_devicegroup_rules.py # Device group rule creation tool
├── scripts/
│   └── encrypt_password.py    # Password encryption utility
├── docs/
│   ├── GET_DEVICE_DETAILS_TOOL.md # Device tool documentation
│   ├── GET_DEVICE_GROUP_DETAILS_TOOL.md # Device group tool documentation
│   ├── GET_DEVICE_METADATA_IDS_TOOL.md # Metadata tool documentation
│   └── ADD_DEVICEGROUP_RULES_TOOL.md # Device group rules tool
├── documentation
│   └── ADD_DEVICEGROUP_RULES_IMPLEMENTATION.md # Implementation details
├── tests/
│   ├── __init__.py
│   ├── test_credentials.py
│   ├── test_get_device_details.py # Device tool tests
│   ├── test_get_device_group_details.py # Device group tool tests
│   ├── test_get_device_metadata_ids.py # Metadata tool tests
│   └── test_add_devicegroup_rules.py # Device group rules tool tests
├── .env.example            # Example environment file
├── .gitignore
├── pyproject.toml
├── requirements.txt
└── README.md

```

Security

Password Encryption

Passwords are encrypted using Fernet (symmetric encryption) from the [cryptography](#) library:

- **Algorithm:** AES-128 in CBC mode with PKCS7 padding
- **Key:** 32-byte URL-safe base64-encoded key
- **Storage:** Encrypted password stored in [.env](#) file
- **Decryption:** Only happens in memory during authentication

Token Management

- **Storage:** Tokens stored in memory only (never persisted to disk)

- **Lifetime:** Conservative 23-hour validity period
- **Refresh:** Automatic refresh on expiration (401 response)
- **Security:** Tokens never logged or exposed

Best Practices

1. **Never commit `.env` file** - It's in `.gitignore` by default
2. **Rotate encryption keys** periodically
3. **Use strong passwords** for SevOne accounts
4. **Enable SSL verification** in production (`SEVONE_VERIFY_SSL=true`)
5. **Restrict file permissions** on `.env` file: `chmod 600 .env`

Logging

The server provides comprehensive logging:

- **INFO:** Normal operations (authentication, API calls)
- **DEBUG:** Detailed request/response data
- **WARNING:** Token expiration, retries
- **ERROR:** Authentication failures, API errors

Configure log level in `.env`:

```
LOG_LEVEL=INFO # Options: DEBUG, INFO, WARNING, ERROR
```

Error Handling

The server handles various error scenarios:

Authentication Errors

```
{
  "success": false,
  "error": "Failed to sign in: HTTP 401",
}
```

```
{
  "message": "Authentication failed - check credentials"
}
```

Token Expiration

- Automatically detected via 401 responses
- Token invalidated and refreshed
- Request retried with new token
- Max 3 retry attempts

API Errors

```
{
  "success": false,
  "error": "API error: 404 - Device not found",
  "message": "Failed to execute GET /api/v3/devices/999"
}
```

Network Errors

```
{
  "success": false,
  "error": "Network error: Connection timeout",
  "message": "Failed to execute POST /api/v3/devices"
}
```

Troubleshooting

"SEVONE_ENCRYPTION_KEY not found in environment"

Solution: Ensure `.env` file exists and contains `SEVONE_ENCRYPTION_KEY`

"Failed to decrypt password"

Solution: Verify the encryption key matches the one used to encrypt the password

"Failed to sign in: HTTP 401"

Solution: Check username and password are correct

"Import errors" when running

Solution: Ensure all dependencies are installed:

```
pip install -r requirements.txt
```

SSL Certificate Errors

Solution: For self-signed certificates, set `SEVONE_VERIFY_SSL=false` in `.env`

Development

Running Tests

```
pytest tests/
```

Code Formatting

```
black src/
```

Linting

```
ruff check src/
```

Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests
5. Submit a pull request

License

[Add your license here]

Support

For issues and questions:

- Create an issue in the repository
- Contact the development team

Custom Bob Modes

This project includes custom Bob modes that provide specialized interfaces to the SevOne API.

1. SevOne API Expert Mode

A comprehensive mode for all SevOne API operations.

What It Does

The **SevOne API Expert** mode:

- Reads the Swagger API documentation automatically
- Understands natural language requests

- Determines the correct API endpoint and method
- Constructs proper request data
- Executes API calls via the MCP server
- Presents results in a user-friendly format

Example Usage

User: "List all devices"

Bob: [Reads Swagger, executes GET /api/v3/devices, shows results in table]

User: "Create a device named Router-01 with IP 192.168.1.1"

Bob: [Reads Swagger, executes POST /api/v3/devices with proper data, confirms creation]

User: "Update location metadata for device 123 to 'Data Center A'"

Bob: [Executes metadata update, confirms success]

Setup

The custom mode is defined in `.bob/custom_modes.yaml` and will be automatically available in Bob once the MCP server is configured.

Documentation

See [CUSTOM_MODE_GUIDE.md](#) for:

- Detailed usage instructions
- Example interactions
- Common use cases
- Troubleshooting tips
- Best practices

2. SevOne Device Query Mode

A specialized mode focused on device queries using the `get_device_details` tool.

What It Does

The **SevOne Device Query** mode:

- Provides a simple, focused interface for device queries
- Accepts device names, IDs, or device group IDs
- Supports fuzzy matching with wildcards
- Can include device metadata
- Presents results in clear, organized formats

Example Usage

```
User: "Find device router1"
Bob: [Queries device by name, shows details]

User: "Show me all routers"
Bob: [Uses fuzzy match with "router*", lists all matching devices]

User: "Get device with ID 123"
Bob: [Queries by device ID, shows details]

User: "List devices in group 10"
Bob: [Queries by device group ID, shows all devices in group]
```

Setup

The mode is defined in `.bob/custom_modes.yaml` and will be automatically available in Bob once the MCP server is configured.

Documentation

See [SEVONE_DEVICE_QUERY_MODE.md](#) for:

- Detailed usage instructions
- Query examples
- Parameter reference
- Tips for best results

Available Tools Documentation

- [get_device_details Tool](#) - Query device information by names, IDs, or group IDs
- [get_device_group_details Tool](#) - Query device group information by IDs or paths
- [get_device_metadata_ids Tool](#) - Get metadata namespace and attribute IDs

- [add_devicegroup_rules Tool](#) - Create device group rules for automatic device assignment

Custom Modes Documentation

- [SevOne Device Query Mode](#) - Specialized mode for device queries

Changelog

Version 1.3.0 (2026-05-25)

- Added [add_devicegroup_rules](#) tool for creating device group rules
- Support for 9 different rule types (metadata, name, IP, SNMP attributes, etc.)
- Regular expression support for flexible device matching
- Comprehensive validation and error handling
- Full test coverage with 7 test cases
- Complete documentation with usage examples

Version 1.2.0 (2026-05-19)

- Added **SevOne Device Query** custom Bob mode
- Specialized interface for device queries
- Natural language device search
- Comprehensive mode documentation

Version 1.1.0 (2026-05-19)

- Added [get_device_details](#) tool for convenient device queries
- Support for querying by device names, IDs, or group IDs
- Fuzzy matching with wildcards
- Optional metadata inclusion
- Comprehensive tool documentation

Version 1.0.0 (2024)

- Initial release
- Secure credential management with encryption
- Automatic token management
- Generic API endpoint tool
- Custom Bob mode for natural language API access
- Support for both stdio and SSE transports
- Comprehensive error handling
- Full logging support